

Raster Graphics Algorithms for Drawing 2-D Primitives

2.1 Introduction

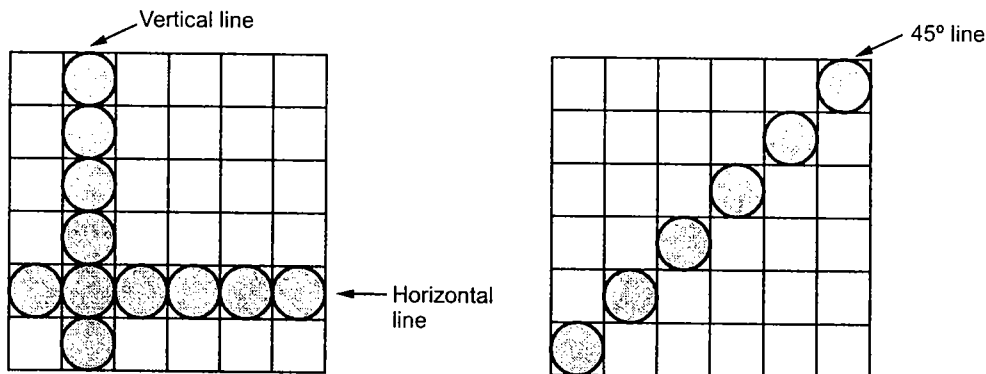
In the previous chapter we have seen that the raster scan graphics devices require special procedures for displaying graphics objects such as line, circle, polygons, curves and even characters. In this chapter we will examine the procedures for line, circle and character generation.

2.2 Basic Concepts in Line Drawing

Before discussing specific line drawing algorithms it is useful to note the general requirements for such algorithms. These requirements specify the desired characteristics of line.

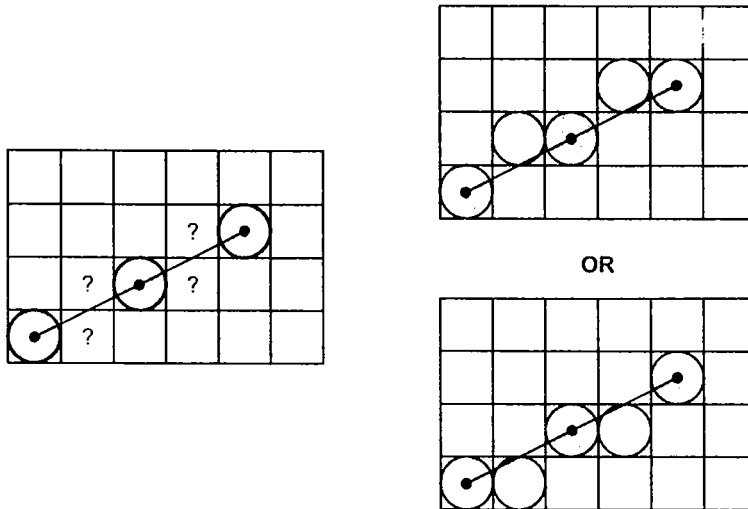
- The line should appear as a straight line and it should start and end accurately.
- The line should be displayed with constant brightness along its length independent of its length and orientation.
- The line should be drawn rapidly.

Let us see the different lines drawn in Fig. 2.1.



(a) Vertical and Horizontal lines

(b) 45° line



(c) Line with other orientation

Fig. 2.1

As shown in Fig. 2.1 (a), horizontal and vertical lines are straight and have same width. The 45° line is straight but its width is not constant. On the other hand, the line with any other orientation is neither straight nor has same width. Such cases are due to the finite resolution of display and we have to accept approximate pixels in such situations, shown in Fig. 2.1 (c).

The brightness of the line is dependent on the orientation of the line. We can observe that the effective spacing between pixels for the 45° line is greater than for the vertical and horizontal lines. This will make the vertical and horizontal lines appear brighter than the 45° line. Complex calculations are required to provide equal brightness along lines of varying length and orientation. Therefore, to draw line rapidly some compromises are made such as

- Calculate only an approximate line length
- Reduce the calculations using simple integer arithmetic
- Implement the result in hardware or firmware

2.3 Line Drawing Algorithms

Considering the assumptions made in the previous section most line drawing algorithms use incremental methods. In these methods line starts with the starting point. Then a fix increment is added to the current point to get the next point on the line. This is continued till the end of line. Let us see the incremental algorithm.

Incremental Algorithm

1. CurrPosition = Start
Step = Increment
2. if ($| \text{CurrPosition} - \text{End} | < \text{Accuracy}$) then go to step 5

[This checks whether the current position is reached upto approximate end point. If yes, line drawing is completed.]

if (CurrPosition < End) then go to step 3

[Here start < End]

if (CurrPosition > End) then go to step 4

[Here start > End]

3. CurrPosition = CurrPosition + Step

go to step 2

4. CurrPosition = CurrPosition - Step

go to step 2

5. Stop.

In the following sections we discuss the line rasterizing algorithms based on the incremental algorithm.

2.3.1 Digital Differential Analyzer

We know that the slope of a straight line is given as

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1} \quad \dots (2.1)$$

The above differential equation can be used to obtain a rasterized straight line. For any given x interval Δx along a line, we can compute the corresponding y interval Δy from equation 2.1 as

$$\Delta y = \frac{y_2 - y_1}{x_2 - x_1} \Delta x \quad \dots (2.2)$$

Similarly, we can obtain the x interval Δx corresponding to a specified Δy as

$$\Delta x = \frac{x_2 - x_1}{y_2 - y_1} \Delta y \quad \dots (2.3)$$

Once the intervals are known the values for next x and next y on the straight line can be obtained as follows

$$\begin{aligned} x_{i+1} &= x_i + \Delta x \\ &= x_i + \frac{x_2 - x_1}{y_2 - y_1} \Delta y \end{aligned} \quad \dots (2.4)$$

and

$$\begin{aligned} y_{i+1} &= y_i + \Delta y \\ &= y_i + \frac{y_2 - y_1}{x_2 - x_1} \Delta x \end{aligned} \quad \dots (2.5)$$

The equations 2.4 and 2.5 represent a recursion relation for successive values of x and y along the required line. Such a way of rasterizing a line is called a **digital differential analyzer** (DDA). For simple DDA either Δx or Δy , whichever is larger, is chosen as one raster unit, i.e.

if $|\Delta x| \geq |\Delta y|$ then

else $\Delta x = 1$
 $\Delta y = 1$

With this simplification, if $\Delta x = 1$ then

we have $y_{i+1} = y_i + \frac{y_2 - y_1}{x_2 - x_1}$ and

$$x_{i+1} = x_i + 1$$

If $\Delta y = 1$ then

we have

$$y_{i+1} = y_i + 1 \text{ and}$$

$$x_{i+1} = x_i + \frac{x_2 - x_1}{y_2 - y_1}$$

Let us see the digital differential analyzer (DDA) routine for rasterizing a line

DDA Line Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.
 [if equal then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. if $(\Delta x \geq \Delta y)$ then
 length = Δx
 else
 length = Δy
 end if
4. $\Delta x = (x_2 - x_1) / \text{length}$
 $\Delta y = (y_2 - y_1) / \text{length}$
 [This makes either Δx or Δy equal to 1 because length is either $|x_2 - x_1|$ or $|y_2 - y_1|$. Therefore, the incremental value for either x or y is one.]
5. $x = x_1 + 0.5 * \text{Sign}(\Delta x)$
 $y = y_1 + 0.5 * \text{Sign}(\Delta y)$
 [Here, Sign function makes the algorithm work in all quadrant. It returns - 1, 0, 1 depending on whether its argument is < 0 , $= 0$, > 0 respectively. The factor 0.5 makes it possible to round the values in the integer function rather than truncating them.]
6. $i = 1$ [Begins the loop, in this loop points are plotted]
 While $(i \leq \text{length})$
 {
 Plot (Integer (x), Integer (y))
 $x = x + \Delta x$
 $y = y + \Delta y$
 $i = i + 1$

7. Stop

Let us see few examples to illustrate this algorithm.

Ex. 2.1 : Consider the line from (0, 0) to (4, 6). Use the simple DDA algorithm to rasterize this line

Sol. : Evaluating steps 1 to 5 in the DDA algorithm we have

$$x_1 = 0$$

$$y_1 = 0$$

$$x_2 = 4$$

$$y_2 = 6$$

$$\therefore \text{Length} = |y_2 - y_1| = 6$$

$$\therefore \Delta x = |x_2 - x_1| / \text{length} \\ = \frac{4}{6}$$

$$\text{and } \Delta y = |y_2 - y_1| / \text{length} \\ = 6 / 6 = 1$$

Initial value for

$$x = 0 + 0.5 * \text{Sign}\left(\frac{4}{6}\right) = 0.5$$

$$y = 0 + 0.5 * \text{Sign}(1) = 0.5$$

Tabulating the results of each iteration in the step 6 we get,

i	Plot	x	y
1	(0, 0)	0.5	0.5
2	(1, 1)	1.167	1.5
3	(1, 2)	1.833	2.5
4	(2, 3)	2.5	3.5
5	(3, 4)	3.167	4.5
6	(3, 5)	3.833	5.5
		4.5	6.5

Table 2.1

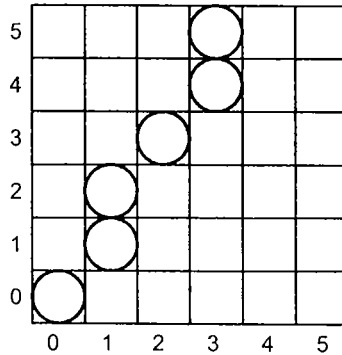


Fig. 2.2 Result for a simple DDA

The results are plotted as shown in the Fig. 2.2. It shows that the rasterized line lies to both sides of the actual line, i.e. the algorithm is **orientation dependent**.

Ex. 2.2 : Consider the line from $(0, 0)$ to $(-6, -6)$. Use the simple DDA algorithm to rasterize this line.

Sol. : Evaluating steps 1 to 5 in the DDA algorithm we have

$$x_1 = 0 \quad y_1 = 0$$

$$x_2 = -6 \quad y_2 = -6$$

$$\therefore \text{Length} = |x_2 - x_1| = |y_2 - y_1| = 6$$

$$\therefore \Delta x = \Delta y = -1$$

Initial values for

$$x = 0 + 0.5 * \text{Sign}(-1) = -0.5$$

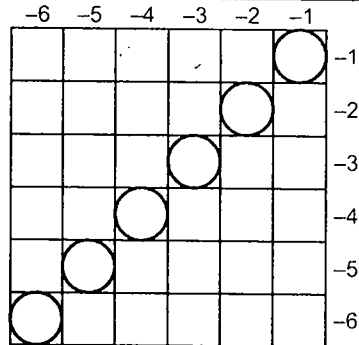
$$y = 0 + 0.5 * \text{Sign}(-1) = -0.5$$

Tabulating the results of each iteration in the step 6 we get,

i	Plot	x	y
		-0.5	-0.5
1	$(-1, -1)$		
		-1.5	-1.5
2	$(-2, -2)$		
		-2.5	-2.5
3	$(-3, -3)$		
		-3.5	-3.5
4	$(-4, -4)$		

5	$(-5, -5)$	-4.5	-4.5
6	$(-6, -6)$	-5.5	-5.5
		-6.5	-6.5

Table 2.2



* -ve pixel values are
with reference to pixel
at the center of screen

Fig. 2.3 Result for a simple DDA

The results are plotted as shown in the Fig. 2.3. It shows that the rasterized line lies on the actual line and it is 45° line.

'C' code for DDA Line Drawing Algorithm

(Softcopy of this program is available at vtubooks.com)

```
#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
float x,y,x1,y1,x2,y2,dx,dy,length;
int i,gd,gm;
clrscr();

/* Read two end points of line
----- */
printf("Enter the value of x1 :\t");
scanf("%f",&x1);
```

```
printf("Enter the value of y1 :\t");
scanf("%f",&y1);
printf("Enter the value of x2 :\t");
scanf("%f",&x2);
printf("Enter the value of y2 :\t");
scanf("%f",&y2);

/* Initialise graphics mode
----- */
detectgraph(&gd,&gm);
initgraph(&gd,&gm,"");

dx=abs(x2-x1);
dy=abs(y2-y1);

if (dx >= dy)
{
    length = dx;
}
else
{
    length = dy;
}

dx = (x2-x1)/length;
dy = (y2-y1)/length;

x = x1 + 0.5; /* Factor 0.5 is added to round the values */
y = y1 + 0.5; /* Factor 0.5 is added to round the values */

i = 1; /* Initialise loop counter */
while(i <= length)
{
    putpixel(x,y,15);
    x = x + dx;
    y = y + dy;
    i = i + 1;
    delay(100); /* Delay is purposely inserted to see
                observe the line drawing process */
}
```



```

    }
    getch();
    closegraph();
}

```

Advantages of DDA Algorithm

1. It is the simplest algorithm and it does not require special skills for implementation.
2. It is a faster method for calculating pixel positions than the direct use of equation $y = mx + b$. It eliminates the multiplication in the equation by making use of raster characteristics, so that appropriate increments are applied in the x or y direction to find the pixel positions along the line path.

Disadvantages of DDA Algorithm

1. Floating point arithmetic in DDA algorithm is still time-consuming.
2. The algorithm is orientation dependent. Hence end point accuracy is poor.

2.3.2 Bresenham's Line Algorithm

Bresenham's line algorithm uses only integer addition and subtraction and multiplication by 2, and we know that the computer can perform the operations of integer addition and subtraction very rapidly. The computer is also time-efficient when performing integer multiplication by powers of 2. Therefore, it is an efficient method for scan-converting straight lines.

The basic principle of Bresenham's line algorithm is to select the optimum raster locations to represent a straight line. To accomplish this the algorithm always increments either x or y by one unit depending on the slope of line. The increment in the other variable is determined by examining the distance between the actual line location and the nearest pixel. This distance is called **decision variable** or the **error**. This is illustrated in the Fig. 2.4.

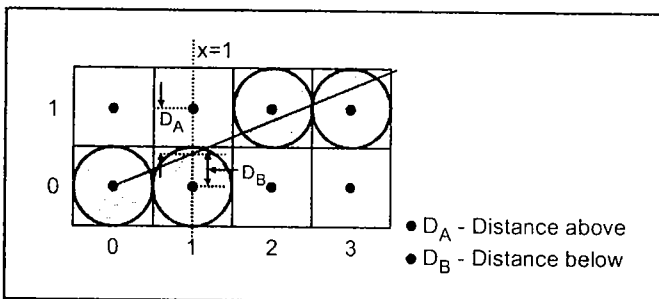


Fig. 2.4

As shown in the Fig. 2.4, the line does not pass through all raster points (pixels). It passes through raster point (0, 0) and subsequently crosses three pixels. It is seen that the intercept of line with the line $x = 1$ is closer to the line $y = 0$, i.e. pixel (1, 0) than to the line $y = 1$ i.e. pixel (1, 1). Hence, in this case, the raster point at (1, 0) better represents the path of the line than

that at (1, 1). The intercept of the line with the line $x = 2$ is close to the line $y = 1$, i.e. pixel (2, 1) than to the line $y = 0$, i.e. pixel (2, 0). Hence, the raster point at (2, 1) better represents the path of the line, as shown in the Fig. 2.4

In mathematical terms error or decision variable is defined as

$$e = D_B - D_A \quad \text{or} \quad D_A - D_B$$

Let us define $e = D_B - D_A$. Now if $e > 0$, then it implies that $D_B > D_A$, i.e., the pixel above the line is closer to the true line. If $D_B < D_A$ (i.e. $e < 0$) then we can say that the pixel below the line is closer to the true line. Thus by checking only the sign of error term it is possible to determine the better pixel to represent the line path.

The error term is initially set as

$$e = 2 \Delta y - \Delta x$$

$$\text{where } \Delta y = y_2 - y_1, \text{ and } \Delta x = x_2 - x_1$$

Then according to value of e following actions are taken.

while ($e \geq 0$)

```
{
    y = y + 1
    e = e - 2 * Δx
}
```

x = x + 1

e = e + 2 * Δy

When $e \geq 0$, error is initialized with $e = e - 2 \Delta x$. This is continued till error is negative. In each iteration y is incremented by 1. When $e < 0$, error is initialized to $e = e + 2 \Delta y$. In both the cases x is incremented by 1. Let us see the Bresenham's line drawing algorithm.

Bresenham's Line Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.
[if equal then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. [Initialize starting point]
 $x = x_1$
 $y = y_1$
4. $e = 2 * \Delta y - \Delta x$
[Initialize value of decision variable or error to compensate for nonzero intercepts]
5. $i = 1$ [Initialize counter]
6. Plot (x, y)
7. while ($e \geq 0$)
{
 $y = y + 1$
 $e = e - 2 * \Delta x$
}
 $x = x + 1$
 $e = e + 2 * \Delta y$
8. $i = i + 1$
9. if ($i \leq \Delta x$) then go to step 6.
10. Stop

Ex. 2.3: Consider the line from (5, 5) to (13, 9). Use the Bresenham's algorithm to rasterize the line.

Sol.: Evaluating steps 1 through 4 in the Bresenham's algorithm we have,

$$\Delta x = |13 - 5| = 8$$

$$\Delta y = |9 - 5| = 4$$

$$x = 5$$

$$y = 5$$

$$e = 2 * \Delta y - \Delta x = 2 * 4 - 8$$

$$= 0$$

Tabulating the results of each iteration in the step 5 through 10.

i	Plot	x	y	e
		5	5	0
1	(5, 5)	6	6	-8
2	(6, 6)	7	6	0
3	(7, 6)	8	7	-8
4	(8, 7)	9	7	0
5	(9, 7)	10	8	-8
6	(10, 8)	11	8	0
7	(11, 8)	12	9	-8
8	(12, 9)	13	9	0
9	(13, 9)	14	10	-8

Table 2.3

The results are plotted as shown in Fig. 2.5.

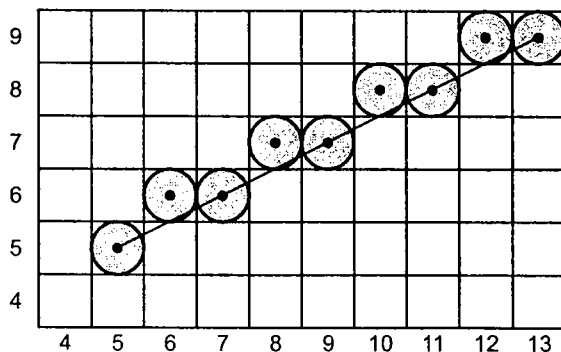


Fig. 2.5

'C' code for Bresenham's Line Drawing Algorithm

(Softcopy of this program is available at vtubooks.com)

```
#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
float x,y,x1,y1,x2,y2,dx,dy,e;
int i,gd,gm;
clrscr();

/* Read two end points of line
----- */
printf("Enter the value of x1 :\t");
scanf("%f",&x1);
printf("Enter the value of y1 :\t");
scanf("%f",&y1);
printf("Enter the value of x2 :\t");
scanf("%f",&x2);
printf("Enter the value of y2 :\t");
scanf("%f",&y2);

/* Initialise graphics mode
----- */
detectgraph(&gd,&gm);
initgraph(&gd,&gm,"");

dx=abs(x2-x1);
dy=abs(y2-y1);

/* Initialise starting point
-----*/
x = x1;
y = y1;

/* Initialise decision variable
----- */
e = 2 * dy-dx;
```

$2dx - dy$

```

i = 1;    /* Initialise loop counter */

do
{
  putpixel(x,y,15);
  while(e >= 0)
  {
    y = y + 1;
    e = e - 2 * dx;
  }
  x = x + 1;
  e = e + 2 * dy;
  i = i + 1;
}
while( i <= dx);
getch();
closegraph();
}

```

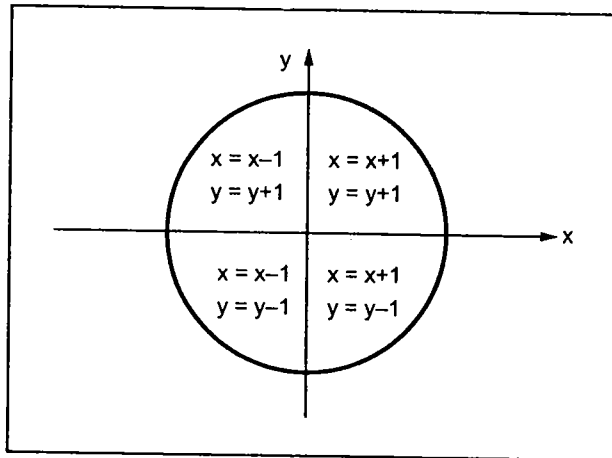


Fig. 2.6 Conditions for generalized Bresenham's algorithm

The Bresenham's algorithm only works for the first octant. The generalized Bresenham's algorithm requires modification for lines lying in the other octants. Such algorithm can be easily developed by considering the quadrant in which the line lies and its slope. When the absolute magnitude of the slope of the line is greater than 1, y is incremented by one and Bresenham's decision variable or error is used to determine when to increment x . The x and y incremental values depend on the quadrant in which the line exists. This is illustrated in Fig. 2.6.

Generalized Bresenham's Algorithm

1. Read the line end point (x_1, y_1) and (x_2, y_2) such that they are not equal.
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. Initialize starting point

$x = x_1$

$y = y_1$

4. $s_1 = \text{Sign}(x_2 - x_1)$
 $s_2 = \text{Sign}(y_2 - y_1)$
[Sign function returns - 1, 0, 1 depending on whether its argument is <0, = 0, > 0 respectively]
5. if $\Delta y > \Delta x$ then
 Exchange Δx and Δy
 Ex_change = 1
else
 Ex_change = 0
end if
 [Interchange Δx and Δy depending on the slope of the line and set Ex_change flag accordingly]
6. $e = 2 * \Delta y - \Delta x$
 [Initialize value of decision variable or error to compensate for nonzero intercept].
7. $i = 1$ [Initialize counter]
8. Plot (x, y)
9. while $(e \geq 0)$
 { if (Ex_change = 1) then
 $x = x + s_1$
 else
 $y = y + s_2$
 end if
 $e = e - 2 * \Delta x$
 }
10. if Ex_change = 1 then
 $y = y + s_2$
else
 $x = x + s_1$
end if
 $e = e + 2 * \Delta y$
11. $i = i + 1$
12. if $(i \leq \Delta x)$ then go to step 8
13. Stop

2.4 Antialiasing of Lines

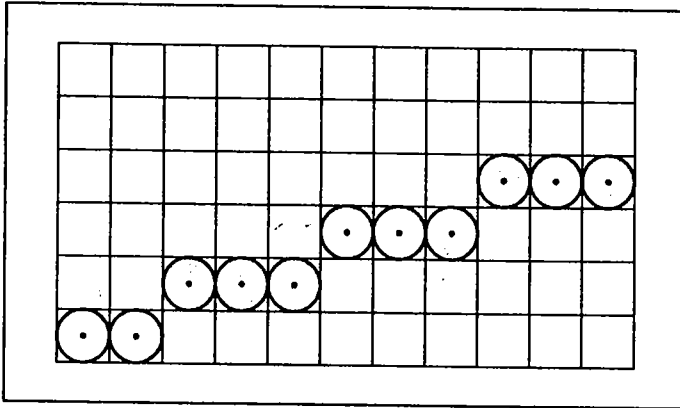


Fig. 2.7 Aliasing effect

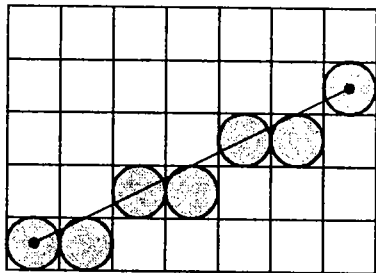
In the line drawing algorithms, we have seen that all rasterized locations do not match with the true line and we have to select the optimum raster locations to represent a straight line. This problem is severe in low resolution screens. In such screens line appears like a stair-step, as shown in the Fig. 2.7. This effect is known as **aliasing**. It is dominant for lines having slopes less than 20° or greater than 70° .

The aliasing effect can be reduced by adjusting intensities of the pixels along the line. The process of adjusting intensities of the pixels along the line to minimize the effect of aliasing is called **antialiasing**.

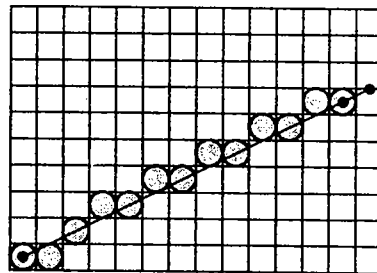
2.5 Methods of Antialiasing

2.5.1 Increasing Resolution

The aliasing effect can be minimized by increasing resolution of the raster display. By increasing resolution and making it twice the original one, the line passes through twice as many column of pixels and therefore has twice as many jags, but each jag is half as large in x and in y direction.



(a)



(b)

Fig. 2.8 Effect on aliasing with increase in resolution

As shown in the Fig. 2.8, line looks better in twice resolution, but this improvement comes at the price of quadrupling the cost of memory, bandwidth of memory and scan-conversion time. Thus increasing resolution is an expensive method for reducing aliasing effect.

2.5.2 Unweighted Area Sampling

We have seen that for sloped lines, many a times the line passes between two pixels. In these cases, line drawing algorithm selects the pixel which is closer to the true line. This step in line drawing algorithms can be modified to perform antialiasing. In antialiasing, instead of picking closest pixel, both pixels are highlighted. However, their intensity values may differ.

In unweighted area sampling, the intensity of pixel is proportional to the amount of line area occupied by the pixel. This technique produces noticeably better results than does setting pixels either to full intensity or to zero intensity.

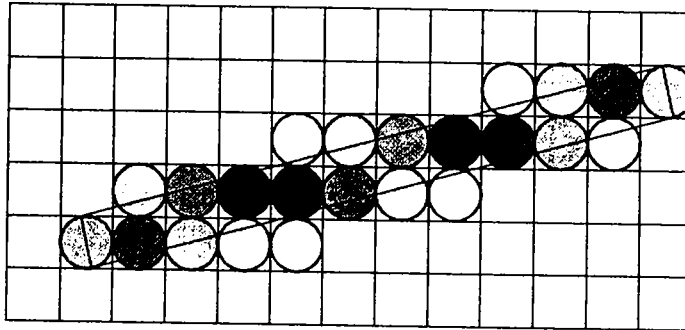


Fig. 2.9 Unweighted area sampling

2.5.3 Weighted Area Sampling

We have seen that in unweighted area sampling equal areas contribute equal intensity, regardless of the distance between the pixel's center and the area; only the total amount of occupied area matters. Thus, a small area in the corner of the pixel contributes just as much as does an equal-sized area near the pixel's center. To avoid this problem even better strategy is used in the weighted area sampling.

In weighted area sampling equal areas contribute unequally i.e. a small area closer to the pixel center has greater intensity than does one at a greater distance. Thus, in weighted area sampling the intensity of the pixel is dependent on the line area occupied and the distance of area from the pixel's center. This is illustrated in Fig. 2.10.

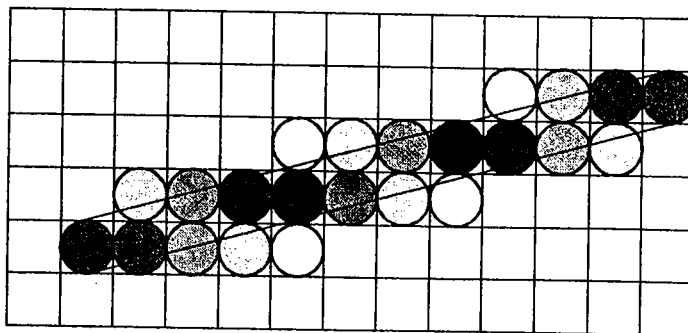


Fig. 2.10 Weighted area sampling

2.7 Basic Concepts in Circle Drawing

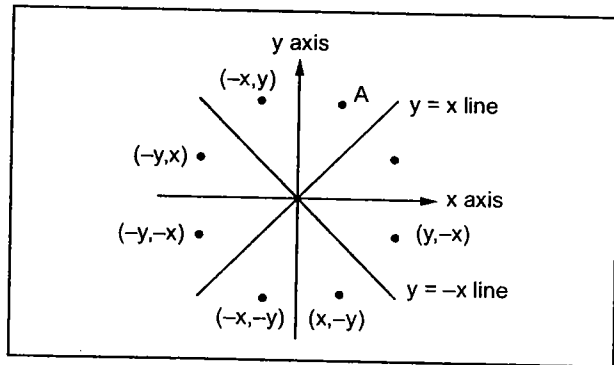


Fig. 2.13 Eight-way symmetry of a circle

A circle is a symmetrical figure. It has eight-way symmetry as shown in the Fig. 2.13. Thus, any circle generating algorithm can take advantage of the circle symmetry to plot eight points by calculating the coordinates of any one point. For example, if point A in the Fig. 2.13 is calculated with a circle algorithm, seven more points could be found just by reflection.

2.8 Representation of a Circle

There are two standard methods of mathematically representing a circle centered at the origin.

2.8.1 Polynomial Method

In this method circle is represented by a polynomial equation.

$$x^2 + y^2 = r^2$$

where x : the x coordinate
 y : the y coordinate
 r : radius of the circle

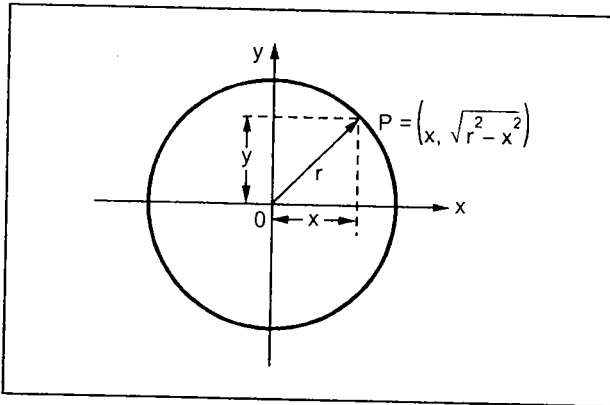


Fig. 2.14 Scan converting circle using polynomial method

Here, polynomial equation can be used to find y coordinate for the known x coordinate. Therefore, the scan converting circle using polynomial method is achieved by stepping x from 0 to $r\sqrt{2}$, and each y coordinate is found by evaluating $\sqrt{r^2 - x^2}$ for each step of x. This generates the 1/8 portion (90° to 45°) of the circle. Remaining part of the circle can be generated just by reflection.

this method for each point both x and r must be squared and x^2 must be subtracted from r^2 ; then the square root of the result must be found out.

2.8.2 Trigonometric Method

In this method, the circle is represented by use of trigonometric functions

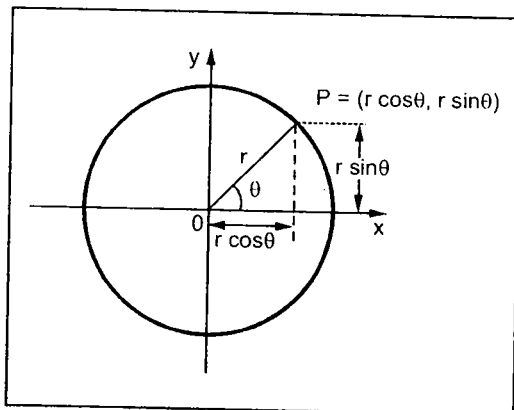


Fig. 2.15 Scan converting circle using trigonometric method

$$x = r \cos \theta \quad \text{and} \quad y = r \sin \theta$$

where θ : current angle
 r : radius of the circle
 x : the x coordinate
 y : the y coordinate

The scan converting circle using trigonometric method is achieved by stepping θ from 0 to $\pi/4$ radians, and each value of x and y is calculated. However, this method is more inefficient than the polynomial method because the computation of the values of $\sin \theta$ and $\cos \theta$ is even more time-consuming than the calculations required in the polynomial method.

2.9 Circle Drawing Algorithms

In this section we are going to discuss two efficient circle drawing algorithms :

- DDA algorithm and
- Bresenham's algorithm
- Midpoint algorithm

2.9.1 DDA Circle Drawing Algorithm

We know that, the equation of circle, with origin as the center of the circle is given as

$$x^2 + y^2 = r^2$$

The digital differential analyser algorithm can be used to draw the circle by defining circle as a differential equation. It is as given below

$$2x dx + 2y dy = 0 \quad \text{where } r \text{ is constant}$$

$$\therefore x dx + y dy = 0$$

$$\therefore y dy = -x dx$$

$$\therefore \frac{dy}{dx} = \frac{-x}{y}$$

From above equation, we can construct the circle by using incremental x value, $\Delta x = \epsilon y$ and incremental y value, $\Delta y = -\epsilon x$, where ϵ is calculated from the radius of the circle as given below

$$2^{n-1} \leq r < 2^n \quad r : \text{radius of the circle}$$

$$\epsilon = 2^{-n}$$

For example, if $r = 50$ then $n = 6$ so that $32 \leq 50 < 64$

$$\begin{aligned} \therefore \epsilon &= 2^{-6} \\ &= 0.0156 \end{aligned}$$

Applying these incremental steps we have,

$$x_{n+1} = x_n + \epsilon y_n$$

$$y_{n+1} = y_n - \epsilon x_n$$

The points plotted using above equations give the spiral instead of the circle. To get the circle we have to make one correction in the equation; we have to replace x_n by x_{n+1} in the equation of y_{n+1} .

Therefore, now we have

$$x_{n+1} = x_n + \epsilon y_n$$

$$y_{n+1} = y_n - \epsilon x_{n+1}$$

Algorithm

1. Read the radius (r), of the circle and calculate value of ϵ
2. $\text{start_x} = 0$
 $\text{start_y} = r$
3. $x_1 = \text{start_x}$
 $y_1 = \text{start_y}$

4. do

```
{  $x_2 = x_1 + \epsilon y_1$ 
```

```
   $y_2 = y_1 - \epsilon x_2$ 
```

```
  [  $x_2$  represents  $x_{n+1}$  and  $x_1$  represents  $x_n$  ]
```

```
  Plot (int (  $x_2$  ), int (  $y_2$  ))
```

```
   $x_1 = x_2$ ;
```

```
   $y_1 = y_2$ ;
```

```
  [Reinitialize the current point ]
```

```
} while (  $y_1 - \text{start}_y < \epsilon$  or (  $\text{start}_x - x_1 > \epsilon$  )
```

```
  [check if the current point is the starting point or not. If current  
  point is not starting point repeat step 4 ; otherwise stop]
```

5. Stop.

'C' code for DDA Circle Drawing Algorithm

(Softcopy of this program is available at vtubooks.com)

```
#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
float x1,y1,x2,y2,startx,starty,epsilon;
int gd,gm,i,val;
int r;
clrscr();

/* Read two end points of line
----- */
printf("Enter the radius of a circle :");
scanf("%d",&r);

/* Initialise graphics mode
----- */
detectgraph(&gd,&gm);
initgraph(&gd,&gm,"");

/* Initialise starting point
----- */
x1=r*cos(0);
```

```

y1=r*sin(0);
startx = x1;
starty = y1;

/*Calculations for epsilon
-----*/
i=0;
do
{
val = pow(2,i);
i++;
}while(val<r);
epsilon = 1/pow(2,i-1);

do
{
x2= x1 + y1*epsilon;
y2 = y1 - epsilon*x2;
putpixel(200+x2,200+y2,15);

/* Reinitialise the current point
----- */
x1=x2;
y1=y2;
delay(1000); /* Delay is purposely inserted to see
               observe the line drawing process */
}
while( (y1 - starty ) < epsilon || (startx - x1) > epsilon);
getch();
closegraph();
}

```

2.9.2 Bresenham's Circle Drawing Algorithm

The Bresenham's circle drawing algorithm considers the eight-way symmetry of the circle to generate it. It plots $1/8^{\text{th}}$ part of the circle, i.e. from 90° to 45° , as shown in the Fig. 2.16. As circle is drawn from 90° to 45° , the x moves in positive direction and y moves in the negative direction.

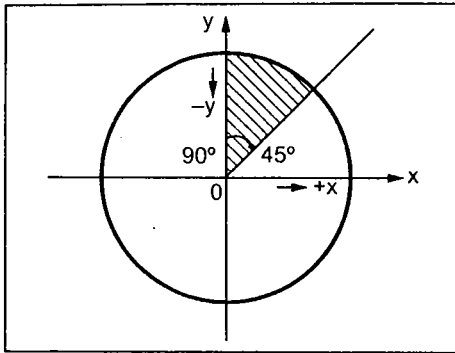


Fig. 2.16 1/8 part of circle

To achieve best approximation to the true circle we have to select those pixels in the raster that fall the least distance from the true circle. Refer Fig. 2.17. Let us observe the 90° to 45° portion of the circle. It can be noticed that if points are generated from 90° to 45°, each new point closest to the true circle can be found by applying either of the two options :

- Increment in positive x direction by one unit or
- Increment in positive x direction and negative y direction both by one unit

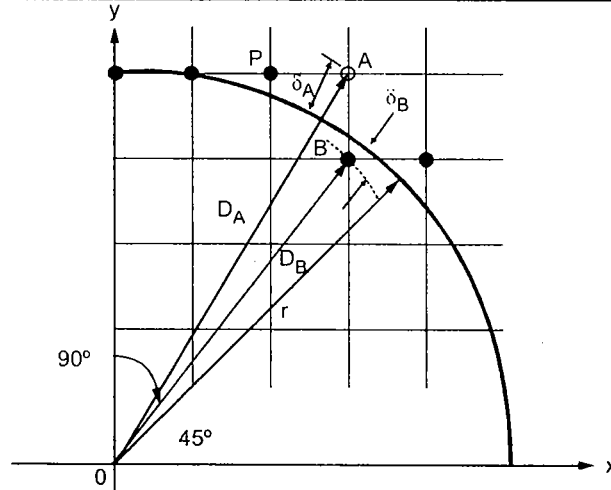


Fig. 2.17 Scan conversion with Bresenham's algorithm

Let us assume point P in Fig. 2.17 as a last scan converted pixel. Now we have two options either to choose pixel A or pixel B. The closer pixel amongst these two can be determined as follows

The distances of pixels A and B from the origin are given as

$$D_A = \sqrt{(x_{i+1})^2 + (y_i)^2} \quad \text{and}$$

$$D_B = \sqrt{(x_{i+1})^2 + (y_i - 1)^2}$$

Now, the distances of pixels A and B from the true circle are given as

$$\delta_A = D_A - r \quad \text{and} \quad \delta_B = D_B - r$$

However, to avoid square root term in derivation of decision variable, i.e. to simplify the computation and to make algorithm more efficient the δ_A and δ_B are defined as

$$\delta_A = D_A^2 - r^2 \quad \text{and}$$

$$\delta_B = D_B^2 - r^2$$

From Fig. 2.17, we can observe that δ_A is always positive and δ_B always negative. Therefore, we can define **decision variable** d_i as

$$d_i = \delta_A + \delta_B$$

and we can say that, if $d_i < 0$, i.e., $\delta_A < \delta_B$ then only x is incremented; otherwise x is incremented in positive direction and y is incremented in negative direction. In other words we can write,

For $d_i < 0$, $x_{i+1} = x_i + 1$ and

For $d_i \geq 0$, $x_{i+1} = x_i + 1$ and $y_{i+1} = y_i - 1$

The equation for d_i at starting point, i.e. at $x = 0$ and $y = r$ can be simplified as follows

$$\begin{aligned} d_i &= \delta_A + \delta_B \\ &= (x_i + 1)^2 + (y_i)^2 - r^2 + (x_i + 1)^2 + (y_i - 1)^2 - r^2 \\ &= (0 + 1)^2 + (r)^2 - r^2 + (0 + 1)^2 + (r - 1)^2 - r^2 \\ &= 1 + r^2 - r^2 + 1 + r^2 - 2r + 1 - r^2 \\ &= 3 - 2r \end{aligned}$$

Similarly, the equations for d_{i+1} for both the cases are given as

For $d_i < 0$, $d_{i+1} = d_i + 4x_i + 6$ and

For $d_i \geq 0$, $d_{i+1} = d_i + 4(x_i - y_i) + 10$

Algorithm to plot 1/8 of the circle

1. Read the radius (r) of the circle.
2. $d = 3 - 2r$
[Initialize the decision variable]
3. $x = 0, y = r$
[Initialize starting point]
4. do
 - {
 - plot (x, y)
 - if ($d < 0$) then
 - {
 - $d = d + 4x + 6$
 - }
 - else
 - { $d = d + 4(x - y) + 10$
 - $y = y - 1$
 - }
 - $x = x + 1$
 - } while ($x < y$)
5. Stop

The remaining part of circle can be drawn by reflecting point about y axis, x axis and about origin as shown in Fig. 2.18.

Therefore, by adding seven more plot commands after the plot command in the step 4 of the algorithm, the circle can be plotted. The remaining seven plot commands are :

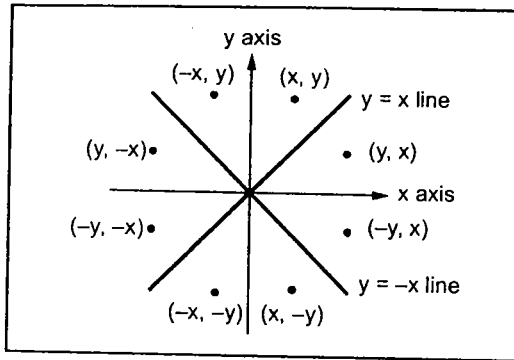


Fig. 2.18 Eight-way symmetry of the circle

plot (y, x)
 plot ($y, -x$)
 plot ($x, -y$)
 plot ($-x, -y$)
 plot ($-y, -x$)
 plot ($-y, x$) and
 plot ($-x, y$)

'C' code for Bresenham's Circle Drawing Algorithm

(Softcopy of this program is available at vtubooks.com)

```

#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
float d;
int gd,gm,x,y;
int r;
clrscr();

/* Read the radius of the circle
-----*/
printf("Enter the radius of a circle :");
scanf("%d",&r);

/* Initialise graphics mode
-----*/
detectgraph(&gd,&gm);
initgraph(&gd,&gm,"");

/* Initialise starting points
-----*/
x = 0;
y = r;
  
```



```

/* initialise the decision variable
-----*/
d = 3 - 2 * r;

do
{
    putpixel(200+x,200+y,15);
    putpixel(200+y,200+x,15);
    putpixel(200+y,200-x,15);
    putpixel(200+x,200-y,15);
    putpixel(200-x,200-y,15);
    putpixel(200-y,200-x,15);
    putpixel(200-y,200+x,15);
    putpixel(200-x,200+y,15);
    if (d <= 0)
    {
        d = d + 4*x + 6;
    }
    else
    {
        d = d + 4*(x-y) + 10;
        y = y - 1;
    }
    x = x + 1;
    delay(1000); /* Delay is purposely inserted to see
                  observe the line drawing process */
}
while(x < y);

getch();
closegraph();
}

```

2.9.3 Midpoint Circle Drawing Algorithm

The midpoint circle drawing algorithm also uses the eight-way symmetry of the circle to generate it. It plots $1/8$ part of the circle, i.e. from 90° to 45° , as shown in the Fig. 2.19. As circle is drawn from 90 to 45° , the x moves in the positive direction and y moves in the negative direction. To draw a $1/8$ part of a circle we take unit steps in the positive x direction and make use of decision parameter to determine which of the two possible y positions is closer to the circle path at each step. The Fig. 2.19 shows the two possible y positions (y_i and $y_i + 1$) at sampling position $x_i + 1$. Therefore, we have to determine whether the pixel at position

$(x_i + 1, y_i)$ or at position $(x_i + 1, y_i - 1)$ is closer to the circle. For this purpose decision parameter is used. It uses the circle function ($f_{\text{circle}}(x, y) = x^2 + y^2 - r^2$) evaluated at the midpoint between these two pixels.

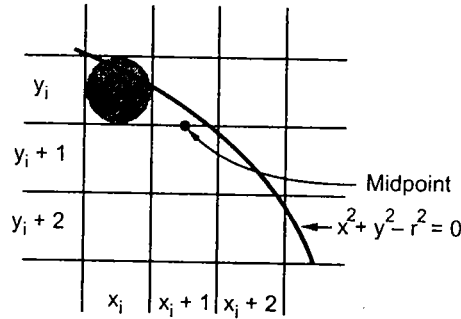


Fig. 2.19 Decision parameter to select correct pixel in circle generation algorithm

$$\begin{aligned}
 d_i &= f_{\text{circle}}(x_i + 1, y_i - \frac{1}{2}) \\
 &= (x_i + 1)^2 + \left(y_i - \frac{1}{2}\right)^2 - r^2 \\
 &= (x_i + 1)^2 + y_i^2 - y_i + \frac{1}{4} - r^2 \quad \dots (2.6)
 \end{aligned}$$

If $d_i < 0$, this midpoint is inside the circle and the pixel on the scan line y_i is closer to the circle boundary. If $d_i \geq 0$, the midposition is outside or on the circle boundary, and $y_i - 1$ is closer to the circle boundary. The incremental calculation can be determined to obtain the successive decision parameters. We can obtain a recursive expression for the next decision parameter by evaluating the circle function at sampling position $x_{i+1} + 1 = x_i + 2$.

$$\begin{aligned}
 d_{i+1} &= f_{\text{circle}}(x_{i+1} + 1, y_{i+1} - \frac{1}{2}) \\
 &= [(x_i + 1) + 1]^2 + \left(y_{i+1} - \frac{1}{2}\right)^2 - r^2 \\
 &= (x_i + 1)^2 + 2(x_i + 1) + 1 + y_{i+1}^2 - (y_{i+1}) + \frac{1}{4} - r^2 \quad \dots (2.7)
 \end{aligned}$$

Looking at equations 2.6 and 2.7 we can write

$$d_{i+1} = d_i + 2(x_i + 1) + (y_{i+1}^2 - y_i^2) - (y_{i+1} - y_i) + 1$$

where y_{i+1} is either y_i or y_{i-1} , depending on the sign of d_i .

If d_i is negative,

$$y_{i+1} = y_i$$

$\therefore$

$$d_{i+1} = d_i + 2(x_i + 1) + 1$$

$$= d_i + 2x_{i+1} + 1$$

$\dots (2.8)$

If d_i is positive,

$$y_{i+1} = y_{i-1}$$

$\therefore$

$$d_{i+1} = d_i + 2(x_i + 1) + 1 - 2y_{i+1}$$

$\dots (2.9)$

The terms $2x_{i+1}$ and $-2y_{i+1}$ in equations (2.8) and (2.9) can be incrementally calculated as

$$2x_{i+1} = 2x_i + 2$$

$$2y_{i+1} = 2y_i - 2$$

The initial value of decision parameter can be obtained by evaluating circle function at the start position $(x_0, y_0) = (0, r)$.

$$\begin{aligned} d_0 &= f_{\text{circle}} \left((0+1)^2 + \left(r - \frac{1}{2} \right)^2 - r^2 \right) \\ &= 1 + \left(r - \frac{1}{2} \right)^2 - r^2 \\ &= 1.25 - r \end{aligned}$$

Algorithm

1. Read the radius (r) of the circle
2. Initialize starting position as
 $x = 0$
 $y = r$
3. Calculate initial value of decision parameter as
 $P = 1.25 - r$
4. do
 - { plot (x, y)
 - if (d < 0)
 - { $x = x + 1$
 - $y = y$
 - $d = d + 2x + 1$
 - }
 - else
 - { $x = x + 1$
 - $y = y - 1$
 - $d = d + 2x + 2y + 1$
 - }
- while (x < y)
5. Determine symmetry points
6. Stop.

'C' code for Midpoint Circle Drawing Algorithm

(Softcopy of this program is available at vtubooks.com)

```
#include<stdio.h>
#include<graphics.h>
```

```
#include<math.h>

main()
{
    float p;
    int i,gd,gm,x,y;
    int r;

    /* initialise graphics
    ----- */
    detectgraph(&gd,&gm);
    initgraph(&gd,&gm,"");

    /* Read the radius
    ----- */
    printf("Enter the radius of the circle :");
    scanf("%d",&r);

    x=0;
    y=r;
    p = 1.25 - r;
    do
    {
        putpixel(200+x,200+y,15);
        putpixel(200+y,200+x,15);
        putpixel(200+x,200-y,15);
        putpixel(200+y,200-x,15);
        putpixel(200-x,200-y,15);
        putpixel(200-x,200+y,15);
        putpixel(200-y,200+x,15);
        putpixel(200-y,200-x,15);

        if (p < 0)
        {
            x = x+1;
            y = y;
            p = p + 2*x + 1;
        }
        else
```

```

{
    x= x+1;
    y= y-1;
    p = p + 2*(x-y) + 1;
}
delay(10000);
}
while(x < y);
getch();
closegraph();
}

```

2.10 Ellipse Drawing Algorithm

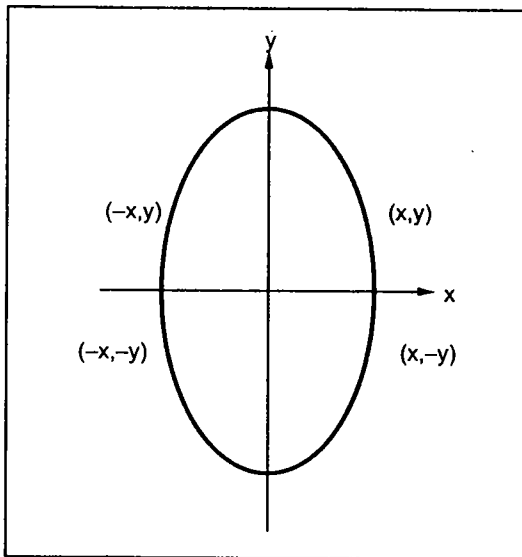


Fig. 2.20 Four way symmetry of ellipse

The midpoint ellipse drawing algorithm uses the four way symmetry of the ellipse to generate it. The Fig. 2.20 shows the four-way symmetry of ellipse. This approach is similar to that used in displaying a raster circle. Here, the quadrant of the ellipse is divided into two regions. The Fig. 2.21 shows the division of the first quadrant according to the slope of an ellipse with $r_x < r_y$. As ellipse is drawn from 90° to 0° , the x moves in the positive direction and y moves in the negative direction, and ellipse passes through two regions. It is important to note that while processing first quadrant we have to take steps in the x direction where the slope of the curve has a magnitude less than 1 (for region 1) and to take steps in the y direction where the slope has a magnitude greater than 1 (for region 2).

Like circle function, the ellipse function $f_{\text{ellipse}}(x, y) (r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2)$ serves as the decision parameter in the midpoint algorithm. At each sampling position, the next pixel along the ellipse path is selected according to the sign of the ellipse function evaluated at midpoint between the two candidate pixels $(x_i + 1, y_i)$ or $(x_i + 1, y_i - 1)$ for region 1 and $(x_i, y_i - 1)$ or $(x_i + 1, y_i - 1)$ for region 2).

Starting at $(0, r_y)$ we have to take unit steps in the x direction until we reach the boundary between region 1 and region 2. Then we have to switch to unit steps in the y direction over the remainder of the curve in the first quadrant. To check for boundary point between region 1 and region 2 we have to test the value of the slope of the curve at each step. The slope of the ellipse at each step is given as

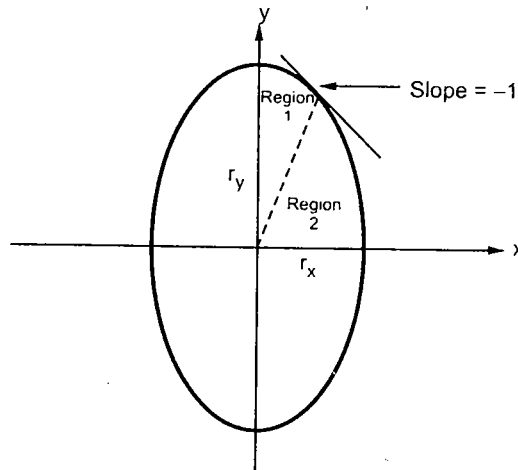


Fig. 2.21 Ellipse processing regions

$$\frac{dy}{dx} = -\frac{2r_y^2 x}{2r_x^2 y}$$

At the boundary point between region 1 and region 2, $dy/dx = -1$ and

$$2r_y^2 x = 2r_x^2 y$$

Therefore, when

$$2r_y^2 x \geq 2r_x^2 y$$

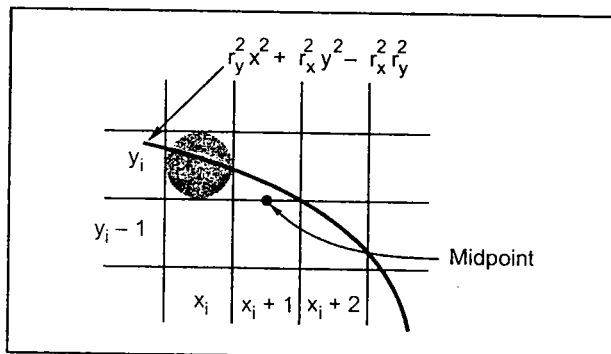


Fig. 2.22

We have to switch to unit steps in the y direction over the remainder of the curve in the first quadrant. The Fig. 2.22 shows the midpoint between the two candidate pixels at sampling position $x_i + 1$ in the first region. The next position along the ellipse path can be evaluated by decision parameter at this midpoint.

$$\begin{aligned} d_{11} &= f_{\text{ellipse}}\left(x_i + 1, y_i - \frac{1}{2}\right) \\ &= r_y^2 (x_i + 1)^2 + r_x^2 \left(y_i - \frac{1}{2}\right)^2 - r_x^2 r_y^2 \end{aligned}$$

If $d_{11} < 0$, the midpoint is inside the ellipse and the pixel on scan line y_i is closer to the ellipse boundary. If $d_{11} \geq 0$, the midpoint is outside or on the ellipse boundary, and the pixel on the scan line $y_i - 1$ is closer to the ellipse boundary.

The incremental calculation of decision parameter of region 1 can be given as

$$\begin{aligned} d_{1i+1} &= f_{\text{ellipse}} \left(x_{i+1} + 1, y_{i+1} - \frac{1}{2} \right) \\ &= r_y^2 [(x_i + 1) + 1]^2 + r_x^2 \left(y_{i+1} - \frac{1}{2} \right)^2 - r_x^2 r_y^2 \end{aligned}$$

Substituting value of d_{1i} in above expression we get,

$$d_{1i+1} = d_{1i} + 2r_y^2 (x_i + 1) + r_y^2 + r_x^2 \left[\left(y_{i+1} - \frac{1}{2} \right)^2 - \left(y_i - \frac{1}{2} \right)^2 \right]$$

where y_{i+1} is either y_i or $y_i - 1$, depending on the sign of d_{1i} .

If d_{1i} is negative, i.e. $d_{1i} < 0$, $y_{i+1} = y_i$

$$\therefore d_{1i+1} = d_{1i} + 2r_y^2 x_{i+1} + r_y^2$$

If d_{1i} is positive or zero, i.e. $d_{1i} \geq 0$, $y_{i+1} = y_i - 1$

$$\therefore d_{1i+1} = d_{1i} + 2r_y^2 x_{i+1} + r_y^2 - 2r_x^2 y_{i+1}$$

The terms $2r_y^2 x$ and $2r_x^2 y$ can be incrementally calculated as

$$2r_y^2 x_{i+1} = 2r_y^2 x_i + 2r_y^2 \text{ and}$$

$$2r_x^2 y_{i+1} = 2r_x^2 y_i - 2r_x^2$$

In region 1, the initial value of the decision parameter can be obtained by evaluating the ellipse function at the start position $(x_0, y_0) = (0, r_y)$.

$$\begin{aligned} d_{10} &= f_{\text{ellipse}} \left(1, r_y - \frac{1}{2} \right) \\ &= r_y^2 + r_x^2 \left(r_y - \frac{1}{2} \right)^2 - r_x^2 r_y^2 \\ &= r_y^2 + r_x^2 r_y + \frac{1}{4} r_x^2 \end{aligned}$$

For region 2, we sample at unit steps in the negative y direction, and the midpoint is now taken between horizontal pixels, at each step, as shown in the Fig. 2.22. For this region, the decision parameter is evaluated as

$$\begin{aligned} d_{2i} &= f_{\text{ellipse}} \left(x_i + \frac{1}{2}, y_i - 1 \right) \\ &= r_y^2 \left(x_i + \frac{1}{2} \right)^2 + r_x^2 (y_i - 1)^2 - r_x^2 r_y^2 \end{aligned}$$

If $d_{2i} > 0$, the midpoint is outside the ellipse boundary, and we select the pixel at x_i .

If $d_{2i} \leq 0$, the midpoint is inside or on the ellipse boundary, and we select pixel position x_{i+1} . The incremental decision parameters for region 2 can be given as

$$\begin{aligned}
 d_{2i+1} &= f_{\text{ellipse}}\left(x_{i+1} + \frac{1}{2}, y_{i+1} - 1\right) \\
 &= r_y^2 \left(x_{i+1} + \frac{1}{2}\right)^2 + r_x^2 [(y_i - 1) - 1]^2 - r_x^2 r_y^2
 \end{aligned}$$

Substituting value of expression d_{2i} in above expression we get,

$$d_{2i+1} = d_{2i} - 2r_x^2 (y_i - 1) + r_x^2 + r_y^2 \left[\left(x_{i+1} + \frac{1}{2}\right)^2 - \left(x_i + \frac{1}{2}\right)^2 \right]$$

where x_{i+1} set either to x_i or to $x_i + 1$, depending on the sign of d_{2i} .

In region 2, the initial value of the decision parameter can be obtained by evaluating the ellipse function at the last position in the region 1.

$$\begin{aligned}
 \therefore d_{20} &= f_{\text{ellipse}}\left(x_0 + \frac{1}{2}, y_0 - 1\right) \\
 &= r_y^2 \left(x_0 + \frac{1}{2}\right)^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2
 \end{aligned}$$

Algorithm

1. Read radii r_x and r_y .
2. Initialise starting point as
 $x = 0$
 $y = r_y$
3. Calculate the initial value of decision parameter in region 1 as

$$d_1 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$$

4. Initialize dx and dy as

$$dx = 2 r_y^2 x$$

$$dy = 2 r_x^2 y$$

5. do

{ plot (x, y)

if ($d_1 < 0$)

{

$$x = x + 1$$

$$y = y$$

$$dx = dx + 2 r_y^2$$

$$d_1 = d_1 + dx + r_y^2$$

$$[d_1 = d_1 + 2 r_y^2 x + 2 r_y^2 + r_y^2]$$

}

else

{

$$x = x + 1$$

$$y = y - 1$$

$$dx = dx + 2 r_y^2$$

$$dy = dy - 2 r_x^2$$

$$d_1 = d_1 + dx - dy + r_y^2$$

$$[d_1 = d_1 + 2 r_y^2 x + 2 r_y^2 - (2 r_x^2 y - 2 r_x^2) + r_y^2]$$

} while (dx < dy)

6. Calculate the initial value of decision parameter in region 2 as

$$d_2 = r_y^2 \left(x + \frac{1}{2} \right)^2 + r_x^2 (y - 1)^2 - r_x^2 r_y^2$$

7. do

{ Plot (x, y)

if ($d_2 > 0$)

{ $x = x$

$$y = y - 1$$

$$dy = dy - 2 r_x^2$$

$$d_2 = d_2 - dy + r_x^2$$

$$[d_2 = d_2 - (2 r_x^2 y - 2 r_x^2) + r_x^2]$$

}

else

{

$$x = x + 1$$

$$y = y - 1$$

$$dy = dy - 2 r_x^2$$

$$dx = dx + 2 r_y^2$$

$$d_2 = d_2 + dx - dy + r_x^2$$

$$[d_2 = d_2 + 2 r_y^2 x + 2 r_y^2 - (2 r_x^2 y - 2 r_x^2) + r_y^2]$$

} while (y > 0)

7. Determine symmetrical points in other three quadrants.
8. Stop.

'C' code for Midpoint Ellipse Drawing Algorithm

(Softcopy of this program is available at vtubooks.com)

```
#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
    long  d1,d2;
    int i,gd,gm,x,y;
    long rx, ry, rxsq,rysq,tworxsq,tworysq,dx,dy;

    /* Read the radius x and y
    ----- */
    printf("Enter the  x radius of the ellipse :");
    scanf("%ld",&rx);

    printf("Enter the y radius of the ellipse :");
    scanf("%ld",&ry);

    /* initialise graphics
    ----- */
    detectgraph(&gd,&gm);
    initgraph(&gd,&gm,"");

    rxsq = rx * rx;
    rysq = ry * ry;
    tworxsq = 2 * rxsq;
    tworysq = 2 * rysq;

    x = 0;
    y = ry;
    d1 = rysq - rxsq * ry + (0.25 * rxsq) ;
    dx = tworysq * x;
    dy = tworxsq * y;
    do
    {
```

```
putpixel(200+x,200+y,15);
putpixel(200-x,200-y,15);
putpixel(200+x,200-y,15);
putpixel(200-x,200+y,15);

if (d1 < 0)
{
    x = x + 1;
    y = y;
    dx = dx + tworysq;
    d1 = d1 + dx + rysq;
}
else
{
    x = x + 1;
    y = y - 1;
    dx = dx + tworysq;
    dy = dy - tworxsq;
    d1 = d1 + dx - dy + rysq;
}
delay(10);
}
while( dx < dy);
d2 = rysq* (x+0.5) * (x+0.5)+rxsq*(y-1)*(y-1)-rxsq*rysq ;
do
{
    putpixel(200+x,200+y,15);
    putpixel(200-x,200-y,15);
    putpixel(200+x,200-y,15);
    putpixel(200-x,200+y,15);

    if(d2 > 0)
    {
        x = x;
        y = y -1;
        dy = dy -tworxsq;
        d2 = d2 - dy + rxsq;
    }
    else
    { x = x+1;
      y = y -1;
      dy = dy -tworxsq;
```

```

        dx = dx + tworysq;
        d2 = d2 + dx - dy + rxsq;
    }
    while (y > 0);
    getch();
    closegraph();
}

```

Solved Examples

Ex. 2.4 The turnover of ABC Company for the following divisions is indicated below :

Construction	Rs. 125 Crores
Engineering	Rs. 90 Crores
Shipping	Rs. 530 Crores
Consumer products	Rs 140 crores
Agro-tech	Rs 115 Crores

Write a program to represent this information in a pie-chart.

(May-97)

Sol. : The 'C' code for above program is as given below

```

#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
    float sum=0, comp[5], d0, angle, theta, x, y, xc, yc;
    int i, gd, gm;

    /* initialise graphics
    ----- */
    detectgraph(&gd, &gm);
    initgraph(&gd, &gm, "");

    /* Read the turnover values */
    for(i=0; i<5; i++)
    {
        printf("Enter the turnover value for company%d :", i+1);
        scanf("%f", &comp[i]);
    }
    for(i=0; i<5; i++)
    {

```

```

sum = sum + comp[i];
}
for(i=0;i<5;i++)
{
comp[i] = (comp[i]/sum)*360;
}
xc = 320;
yc = 240;
x = 420;
y = 240;
d0 = 1/(3.2 * (abs(x-xc) + abs(y-yc)));
angle = 0;
theta = 0;
    for(i=0;i<5;i++)
    {
        theta = theta + comp[i];
        while (angle*180/3.142 < theta )
        {
            setcolor(i+1);
            line(xc,yc,x,y);
            x = x - (y-yc) * d0;
            y = y + (x - xc)*d0;
            angle = angle + d0;
            delay(100);
        }
    }
getch();
closegraph();
}

```

Ex. 2.5 Consider the following circuit where the switch, is closed at $t = 0$. Develop a program to display the plot of $V_o(t)$ for $t > 0$.

(May-2000)

Sol.: The 'C' code for above program is as given below

```

#include<stdio.h>
#include<graphics.h>
#include<math.h>

```

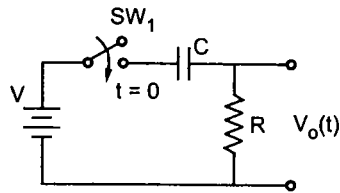


Fig. 2.23

```

main()
{
    float d,t,c,r,vo,v,T;
    int i,gd,gm,x,y;
    detectgraph(&gd,&gm);
    initgraph(&gd,&gm,"");
    printf("Enter the value of battery voltage :");
    scanf("%f",&v);
    printf("Enter the value of capacitor :");
    scanf("%f",&c);
    printf("Enter the value of resistor :");
    scanf("%f",&r);
    t= 0.0;
    /* Draw axis */
    line(50,400,600,400);
    line(100,440,100,50);
    for(i=0;i<450;i++)
    {
        T = -t/(c*r);
        vo = v - (v*(1-exp(T)));
        t = t + 0.001;
        putpixel(100+t*1000,400-vo*(300/v),15);
        delay(1000);
    }
    getch();
    closegraph();
}

```